

Contents

1 Battery pathway (deintercalation electrode characteristics)	1
1.1 What tier 1 computes (MLIP-only, no new calculation plugins)	1
1.2 Method choices locked in	2
1.3 Architecture / file map	2
1.3.1 input.yaml block (all keys optional; module runs without the block)	3
1.4 Status	3
1.5 Tier-2 notes (for when we get there)	4
1.6 Head provenance (2026-07-22)	4

1 Battery pathway (deintercalation electrode characteristics)

A bulk pathway that runs after the gen/csp + phase-diagram stages, as an alternative to the catalysis (surface builder + adsorbates) branch. It takes the stable host structure(s) containing a working ion A (Li, Na, K, Mg, Zn), removes A stepwise, and computes electrode characteristics from the energies $E(x)$ of $A_x \cdot \text{Host}$ on a pseudo-binary convex hull.

Submission reuses the existing schema with **no DB migration**:

```
{"composition": "LiFePO4", "reaction": "battery", "reaction_path": "Li"}
```

reaction_path carries the working ion, and the per-ion step status nests in DBComposition.step_status["battery"] exactly like the per-(reaction, pathway) adsorbates status. Surface builder, adsorbates and the catalysis calculators are skipped for battery submissions (bulk pathway, no slabs).

1.1 What tier 1 computes (MLIP-only, no new calculation plugins)

For each host and working ion:

- **Voltage profile** from the pseudo-binary convex hull over x . Between adjacent hull vertices $x_1 \rightarrow x_2$:
$$V = -[E(x_2) - E(x_1) - (x_2 - x_1) \cdot \mu_A] / (z \cdot (x_2 - x_1) \cdot e)$$
with μ_A = energy/atom of the elemental ion metal (same MLIP, via the existing elemental-reference machinery) and $z = 1$ (Li/Na/K) or 2 (Mg/Zn).
- **Average voltage** over the full range (depends only on the end members).
- **Gravimetric capacity** $Q = n \cdot z \cdot F / (3.6 \cdot M)$ in mAh/g, M = molar mass of the discharged formula unit (standard cathode convention: LiFePO₄ $\rightarrow$ 170 mAh/g).
- **Volumetric capacity** (mAh/cm³) from the discharged-state volume.
- **Specific energy** (Wh/kg) = $Q_{\text{grav}} \times V_{\text{avg}}$.
- **Volume change** between charged and discharged end members (cycling proxy).
- **Charged-endpoint stability**: $e_{\text{above_hull}}$ of the empty host against the chemsys hull the gen path already built. Far above the hull = conversion-like chemistry $\rightarrow$ flagged, not silently reported as intercalation.
- **Framework integrity**: StructureMatcher on the host sublattice (ion removed) charged vs discharged, plus a volume-collapse guard. Detects layer gliding / reconstruction that invalidates the intercalation picture.

All of tier 1 rides on the existing MLIP relax workchains (MACE / UMA / MatterSim, job_type: relax) — zero new AiiDA calculation or parser plugins.

1.2 Method choices locked in

1. **v1 is deintercalation-only.** The submitted formula must contain the working ion (LiCoO₂, NaFePO₄, ...). Insertion into an empty host (TiO₂ + Li) needs interstitial-site search and is a v2 feature.
2. **One common supercell for all x**, built once from the host primitive cell (capped at `supercell_max_atoms`), so every grid point is an exact fraction k/N of the N ion sites — no incommensurate occupancies.
3. **Enumeration is pymatgen-only** (no `enum.x` binary, no `icet` requirement): partial occupancy x on the ion sublattice, orderings ranked by Ewald electrostatics (oxidation states from BVAnalyzer / composition guess); if no oxidation states can be assigned (metallic hosts), fall back to symmetry-distinct sampling with a fixed seed. `icet` enumeration can replace this later without touching the workchain. **Since 2026-07-22 the enumeration runs on a REMOTE CPU queue** (`input.yaml` `battery: enum_code:`, default `sqs_cpu` – the synthesizability `finite_T` code): one bundled job for all hosts, staged as `battery_enum_job.py` -> `aiida.py` with `workchains/battery_enum.py` shipped verbatim via the file namespace (single canonical copy) and collected by ECHOED host `uuid`, never by position. If `enum_code` is not registered in `config.yaml` the workchain falls back to `in-daemon` enumeration with a loud report (local smoke).
4. **Fail loudly.** Framework collapse and hull-unstable charged endpoints are recorded as flags on the DB row; a voltage from a structure that fell apart is never reported as a clean result.
5. **PBE-head caveat.** MACE `omat_pbe` is plain PBE: absolute voltages of TM-oxide redox couples sit systematically low (roughly 0.5–1 V). Fine for *ranking*; publishable absolute voltages need the tier-2 DFT (+U) stage.

1.3 Architecture / file map

piece	file	notes
pure calculator	<code>workchains/batt.py</code>	hull, voltages, capacities, volume change; no AiiDA imports , unit-tested standalone (same family as <code>oer.py/co2rr.py</code>)
enumeration helper	<code>workchains/battery_enum.py</code>	supercell choice + per-x orderings; pure pymatgen, unit-tested standalone
workchain	<code>workchains/battery.py</code>	BatteryWorkChain: <code>setup</code> -> <code>run_enum</code> (remote CPU job) -> <code>inspect_enum</code> -> bundled MLIP relax fan-out -> <code>analyze</code> -> <code>store</code>
remote enum runner	<code>codes/files/battery_enum_job.py</code>	staged as <code>aiida.py</code> on the <code>enum_code</code> code; request/output contracts in its docstring; <code>tests/test_battery_enum_job.py</code>
DB table	<code>db/tables.py::DBBatteryPath</code>	one row per (composition, working_ion, host); results in Postgres, not AiiDA storage (ephemeral)
entry point	<code>setup.json</code>	<code>battery = uvsib.workchains.battery:BatteryWork</code>
main gating	<code>workchains/main.py</code>	<code>should_run_battery</code> on <code>reaction == "battery"</code> ; surface builder + adsorbates skip battery submissions

piece	file	notes
tests	tests/test_batt.py, tests/test_battery_enum.py	run in any env with pymatgen, no AiiDA / no DB
local smoke	tests/smoke_battery_mace.py	standalone LFP/LCO end-to-end with a local MACE (not collected by pytest)

The relax fan-out bundles the working-ion elemental reference through the same `missing_element_references` / `element_reference_entries` machinery the gen and csp paths use, so the anode reference (Li bcc, Na bcc, ...) is relaxed on the same MLIP — methodologically consistent voltages for free.

1.3.1 input.yaml block (all keys optional; module runs without the block)

```
battery:
  model: MACE                # MLIP for the config relaxations (defaults to bulk_relax)
  head: Default
  fmax: 0.05
  max_steps: 200
  n_x_steps: 4               # intermediate compositions between charged and discharged
  max_configs_per_x: 8       # orderings kept per grid point (Ewald-ranked)
  supercell_max_atoms: 128
  max_hosts: 3               # stable_struct candidates to sweep (<= MAX_NUM_BULK)
```

1.4 Status

- ☒ plan agreed (2026-07-21)
- ☒ workchains/batt.py pure calculator + unit tests
- ☒ workchains/battery_enum.py enumeration helper + unit tests
- ☒ BatteryWorkChain + DB table + entry point + main.py gating
- ☒ local end-to-end smoke (2026-07-21, MACE-MP-0 medium, RX 6900 XT; enumerate -> cell+position relax -> batt.py, ~50 s per material): - LiFePO4 (mp-19017): V_{avg} 3.438 V (exp 3.45 - MPtrj carries Fe +U), Q 169.9 mAh/g (theoretical, exact), 584 Wh/kg, dV -5.2% (exp ~ -7%), framework intact; the 3.25-3.60 V staircase around the experimental flat plateau is the finite-supercell ordering artifact, as expected - LiCoO2 (mp-22526): V_{avg} 3.277 V (low vs ~3.9 exp - the documented MLIP voltage caveat for Co redox), Q 273.8 mAh/g (theoretical, exact), 898 Wh/kg, dV +4.6% - correctly POSITIVE (layered c-axis expansion on delithiation, opposite sign to olivine), O3 framework intact - determinism confirmed: repeated runs bit-identical energies
- ☒ tier 2 NEB engine + battery driver (2026-07-21): - codes/files/neb.py — the SHARED (CI-)NEB engine (job_type: "neb" in all four MLIP runners): endpoint pre-relax, IDPP with mic, FIRE, two-stage climbing image, bundled pairs, per-pair fail-loudly records. The future catalysis NEB uses THIS engine with its own driver. - workchains/batt_neb.py — pure battery driver: symmetry-distinct hop enumeration, vacancy/dilute endpoint pairs (identical atom ordering by construction), periodic percolation analysis (offset union-find: e_m at which the hop network wraps the cell in 1/2/3 directions) - BatteryNEBWorkChain + db_battery_neb + main.py gating (battery: neb: enabled, per-ion step_status["battery_neb"]) - smoke (MACE-MP-0 medium, RX 6900 XT): **LiFePO4 in-channel hop d=3.05 Å, Ea 0.392 eV (GGA lit. 0.27-0.55), symmetric single-saddle band, percolation 1D at 0.39 eV / 2D/3D unreachable — the textbook 1D b-channel result reproduced end-to-end in ~80 s**; LiCoO2 checks the 2D in-plane case
- ☒ MP-experimental injection (2026-07-21): experimentally-known MP structures are seeded into GNoME, injected verbatim into the csp/gen relax bundles, and force-included (deduplicated, prepended) into stable_struct — the battery stage is guaranteed to sweep the real polymorphs (olivine vs maricite NaFePO4 case closed). Details: docs/mp_experimental.md
- ☐ deploy: python -m uvsib.db.tables (creates db_battery_path + db_battery_neb), reinstall for the new entry points, submit a battery entry; flip battery: neb: enabled for tier 2

- ❑ tier 2: DFT (+U) verification stage on hull-vertex configurations and NEB TS images (stored in `db_battery_neb.hops` for exactly this)
- ❑ catalysis NEB driver on the shared engine (site-adjacency / reaction pairs — waiting on janK's structure/pathway concept)
- ❑ v2: insertion mode for ion-free hosts (interstitial site search)

1.5 Tier-2 notes (for when we get there)

- **NEB**: one generic MLIP-NEB workchain (images from IDPP interpolation, climbing image, existing relax calcjobs extended with a `job_type: neb`) serves both battery migration barriers and catalysis reaction barriers — do not build two.
- **DFT verification**: statics/short relaxes on the hull vertices only (typically 3-6 structures per host), PBE+U per the `battery_fw` settings; MP2020-style corrections only matter if we mix with MP entries — internal voltages need consistent U, nothing else.
- **Electrolyte window / interface reactivity** (grand-potential hulls): out of scope until someone asks.

1.6 Head provenance (2026-07-22)

The method column ("MACE") cannot distinguish task heads, and mixing heads in one hull/reference set is a silent systematic error (caught when the LiCoO₂ voltages could not be attributed: the export did not record that battery: head: `omat_pbe` produced them while the bulk hull ran `matpes_r2scan`). Now:

- every writer stores `attributes["model_head"]`: structure versions (`add_structures(..., head=)` - `gen/csp/sqs/references/mp_experimental`), slabs (`add_slab`), adsorbate rows (`add_surface_ml_adsorbate`), `db_battery_path`, `db_battery_neb`;
- elemental-reference queries are head-aware and STRICT: `missing_element_references(..., head=)` counts an element as missing unless a reference with that exact head exists (foreign-head references are never silently reused; pre-provenance head-less rows do not match, so each head recomputes its references once - self-healing, one cheap relax);
- the charged-endpoint `e_above_hull` references the BULK head and reports loudly when probe and hull heads differ (cross-head comparison);
- the csp pool stores head-less (with a report) if `bulk_relax` and `MinimaHopping` heads ever diverge - a mixed pool has no single head;
- `export_all.py` exposes `model_head` on surface / adsorbate / battery / battery-NEB records.

No schema change (attributes JSONB existed); deploy = reinstall only.

Different heads per stage are a FEATURE, not a violation (janK, 2026-07-22): the adsorbates branch keeps its own head option (e.g. `oc20_usemppbpe` for adsorption energetics while bulks run `matpes_r2scan`) - that is the right tool per task. Head provenance does not force uniformity; it only guarantees that (a) every stored number says which head produced it and (b) references/hulls are never silently mixed ACROSS heads within one comparison.